

FlexQL

Design Document

Vivek Chaubey · Roll No. 25CS60R65

IIT Kharagpur · github.com/vivekchaubey/IITKGP25/FLEXQL

FlexQL is a SQL-like embedded database server written from scratch in C++17. This document covers the six design areas required for the submission: data storage, indexing, caching, expiration-timestamp handling, multithreading, and additional design decisions.

1. How the Data is Stored

1.1 Row-major disk layout

Every table owns one append-only binary row file on the local filesystem (default root: the directory passed as FLEXQL_STORAGE_ROOT, or a temporary directory when not set). A row is serialised as a fixed-size header followed by a sequence of length-prefixed string fields, one per column, plus a 64-bit *expires_at* timestamp in Unix milliseconds. Because the file is append-only, the byte offset of a row never changes after it is written — this is the property that makes the primary-key hash index reliable.

Row-major layout was chosen deliberately over columnar storage. Every query in this workload either inserts a full row, does a primary-key point lookup that returns a full row, or scans all rows. Columnar layout would only help analytics that project a small subset of columns — none of the current query types benefit from it.

1.2 Table metadata (struct Table)

Each in-memory **Table** object (include/storage/storage.h) tracks:

Field	Type	Purpose
name	std::string	Identifier used as catalog key
schema	vector<ColumnDef>	Column names, types, constraints
rowFilePath	std::string	Absolute path to the row file
rowFileFd / readFileFd	int (fd)	Write fd (kept open) + read fd
rowFileEndOffset	uint64_t	Byte offset of next append
rowCount	uint64_t	Live row counter
primaryKeyIdx	int	Column index of the PK, or -1
primaryKeyIndexPath/Fdstring / int		Disk-backed PK hash file
rwlock	shared_mutex	Per-table read/write lock

1.3 Catalog

The **StorageEngine** holds an *unordered_map<string, unique_ptr<Table>>* protected by a *catalogLock_* (shared_mutex). Table creation and deletion hold an exclusive catalog lock; all other operations take a shared catalog lock and then promote to the per-table lock as needed.

1.4 Column type system

The **ColType** enum (include/common/types.h) defines four types: INT, DECIMAL, VARCHAR, and DATETIME. All values are stored as strings on disk. INT and DECIMAL are parsed to numbers only at comparison time inside the executor's WHERE-clause evaluator. TEXT is accepted as a synonym for VARCHAR during parsing. This keeps the serialiser simple and avoids version-skew problems if the schema changes.

1.5 Write-ahead log (WAL)

The **DurableLog** class (include/storage/durable_log.h) implements an append-only binary log stored at *data/flexql.wal* by default. It is the primary source of truth; the row files are a derived, rebuildable working set. Every mutation follows this sequence:

1. Validate the statement.
2. `DurableLog::beginInsert / beginCreate / beginDelete` – write + fsync a PREPARE record.
3. Apply the change: append to the row file, update the PK index.
4. `DurableLog::commit` – write + fsync a COMMIT marker.
5. Return success to the client.

If step 3 or 4 fails, the local change is rolled back and an error is returned. On restart, *replayDurableLog()* rebuilds every table by replaying only records that have a matching COMMIT marker; any trailing PREPARE without a COMMIT is silently discarded. A large batch insert uses *beginInsertNoSync / commitNoSync* followed by a single explicit *sync()*, reducing fsync calls without weakening the per-batch durability guarantee.

2. Indexing Method

2.1 In-memory HashIndex

The **HashIndex** class (include/index/hash_index.h) wraps an *unordered_map<string, size_t>* that maps a primary-key string value to the row's byte offset in the table row file. It is pre-reserved with a hint of 2^{20} buckets to avoid rehashing during initial load. Operations are $O(1)$ average: *insert*, *lookup*, *remove*, *update*.

The index is only built for tables that declare a PRIMARY KEY column. It is registered in the Executor via *registerIndex(tableName, pkColIdx)* and rebuilt from scratch after WAL replay via *rebuildIndex()*.

2.2 Index spill to disk

The Executor caps the resident index at *maxIndexedRows_*. When a table grows past that limit, the in-memory HashIndex map is cleared. The StorageEngine then falls back to the disk-backed primary-key hash file (stored alongside the row file) for both duplicate-key checks and point lookups. This keeps large tables from exhausting the index heap while avoiding full-table scans for primary-key equality queries.

2.3 Lookup decision tree (executed per WHERE clause)

Condition	Path taken
WHERE targets PK + resident HashIndex present	in-memory unordered_map lookup ($O(1)$)
WHERE targets PK + index has spilled	disk-backed PK hash file lookup
WHERE targets non-PK column, or no WHERE	sequential scan of row file

Secondary indexes are not currently implemented. A non-primary-key WHERE always triggers a full sequential scan. This is a known limitation noted for future work.

3. Caching Strategy

3.1 LRU result cache

The **LRUCache** class (`include/cache/lru_cache.h`) stores full SELECT result sets keyed by a query signature. The internal structure is a doubly-linked list (`std::list<Entry>`) paired with an `unordered_map` from key to list iterator. This gives O(1) get, put, and eviction.

3.2 Capacity limits

Limit	Default	Controlled by
Entry count	1 024	LRUCache constructor arg <code><i>capacity</i></code>
Total bytes	8 MiB	LRUCache constructor arg <code><i>maxBytes</i></code>

When a new result would push either limit over its budget, the least-recently-used entry is evicted first. Large result sets that exceed the byte budget stream through the executor but are not stored in the cache.

3.3 Cache key construction

The static helper `Executor::makeCacheKey(query)` serialises the table name, projected column list, WHERE predicate (column, operator, value), JOIN target and columns, and ORDER BY column into a single string. Two logically identical queries always produce the same key, regardless of whitespace differences in the original SQL, because the key is built from the parsed AST fields rather than the raw SQL text.

3.4 Cache invalidation

On every INSERT, DELETE, or CREATE TABLE, the executor calls `LRUCache::invalidateTable(tableName)`. This iterates the map and removes every entry whose key contains the table name as a prefix component. The strategy is conservative (any write to table T purges all cached queries that touch T) but simple and correct.

3.5 TTL re-check on cache hit

The **CachedResult** struct stores a parallel `vector<int64_t> rowExpiresAt` alongside the row data. When the cache returns a hit, the executor re-filters the result set against `now()` in milliseconds before streaming rows to the client. This means a cached result never returns a row that has since expired, even if it was live when first cached.

4. Handling of Expiration Timestamps

4.1 Data model

Every **Row** struct (`include/common/types.h`) carries an `int64_t expires_at` field in Unix milliseconds. A value of 0 means the row never expires. The field is written to the row file and replayed through the WAL exactly like any other column.

4.2 INSERT syntax

```
INSERT INTO orders VALUES (1, 'Widget', 9.99) EXPIRES 1735689600000;
```

The parser recognises the optional **EXPIRES** `<unix_ms>` clause at the end of an INSERT statement and stores the value in *ParsedQuery::expiresAt*. When the clause is absent, *expiresAt* defaults to 0 (no expiry).

4.3 Enforcement strategy — lazy expiration

Where enforced	Mechanism
Sequential scan	StorageEngine::scanRows skips rows where expires_at > 0 AND expires_at <= now()
PK index lookup	After the row is read by offset, expires_at is checked before returning it
Cache hit	Executor re-filters CachedResult::rowExpiresAt against now() before streaming
Duplicate-key check	An expired row at a PK does not block a new INSERT with the same PK value

Expiry is enforced lazily — there is no background thread that deletes rows. This keeps the write path simple and avoids lock contention from a concurrent sweeper. The trade-off is that expired rows still occupy disk space until a future compaction pass (not yet implemented) rewrites the row file without them.

5. Multithreading Design

5.1 Thread pool

The server creates a fixed thread pool on startup, sized as $\max(4, \text{std::thread::hardware_concurrency}() \times 2)$. Each accepted TCP connection is dispatched to an idle pool thread, which calls *serveClientConnection(clientFd, executor)* (include/server/session.h). The pool prevents unbounded thread creation under high connection load.

5.2 Lock hierarchy

Six distinct locks are used. They are always acquired in the order shown below to prevent deadlock.

#	Lock	Type	Scope / when held
1	StorageEngine::catalogLock_	shared_mutex	Shared: any table access. Exclusive: CREATE/DROP TABLE.
2	Table::rwlock	shared_mutex	Shared: row scans and reads. Exclusive: row writes and clears.
3	Executor::indexMutex_	mutex	All HashIndex insert / lookup / update operations.
4	LRUCache::mutex_	mutex	All cache get / put / invalidate operations.
5	DurableLog::mutex_	mutex	All WAL appends (beginXxx, commit, sync).
6	Executor::mutationMutex_	mutex	Held for the entire duration of every INSERT, DELETE, CREATE.

The *mutationMutex_* at position 6 serialises all writes globally. This means concurrent reads can proceed in parallel, but concurrent writes are queued. The trade-off is accepted because correctness and a clear durability story were higher priorities than maximising write concurrency.

5.3 Read parallelism

Because catalog and table locks are shared mutexes, multiple clients can execute SELECT queries simultaneously without blocking each other, as long as no mutation is in progress. The LRU cache and WAL locks are plain mutexes and will briefly serialise concurrent cache lookups and WAL reads, but these operations are short compared to a disk scan.

5.4 Streaming abort

Row results are sent to the client one frame at a time inside the *flexql_callback* loop. Between frames the session handler checks whether the client has sent an ABORT frame and, if so, stops execution early. This allows a client to cancel a large query without waiting for the full result set to be materialised.

6. Additional Design Decisions

6.1 Network protocol

All TCP messages use a 4-byte big-endian length prefix followed by the payload bytes (max 16 MiB, defined as `FLEXQL_MAX_MSG` in `types.h`). *net::sendMsg* and *net::recvMsg* handle framing. Sockets are configured with `TCP_NODELAY` to reduce latency for small frames.

Direction	Frame prefix	Payload
Client → Server	QUERY\n	SQL text
Client → Server	ABORT	(empty — cancel current stream)
Client → Server	EXIT	(empty — close connection)
Server → Client	HEADER\n	Tab-separated column names
Server → Client	ROW\n	Tab-separated row values
Server → Client	END	(empty — result set complete)
Server → Client	ABORTED	(empty — abort acknowledged)
Server → Client	ERR\n	Error message text

6.2 Incremental streaming

The executor calls *flexql_callback* for each row as it is produced by the storage scan. Rows are sent to the client immediately rather than being accumulated in a buffer first. The exception is ORDER BY and multi-table JOIN: these operations must materialise the full intermediate result before they can sort or merge, so they consume more memory for large result sets.

6.3 Batched INSERT optimisation

A single QUERY frame may contain multiple value tuples: *INSERT INTO t VALUES (1,'a'),(2,'b'),...;* The parser populates *ParsedQuery::insertRows* as a vector of tuples. The executor holds *mutationMutex_* once for the whole batch, writes all rows to the row file in sequence, and uses *beginInsertNoSync* for intermediate records with a single final *sync()*. This cuts both lock-acquisition

overhead and fsync calls dramatically for bulk loads.

6.4 Persistent write file descriptor

Each Table keeps its write-side row file descriptor (*rowFileFd*) open across multiple inserts rather than opening and closing it for each row. This avoids repeated kernel open/close syscalls. A separate read-side descriptor (*readFileFd*) is used for scans so that the read position does not interfere with appends.

6.5 Snapshot persistence

In addition to WAL replay, the *persistence* namespace (include/storage/persistence.h) provides *saveSnapshot* and *loadSnapshot* helpers that serialise the current storage state to a compact binary file. Snapshots allow faster cold-start recovery than replaying a long WAL from the beginning.

6.6 Supported SQL subset

Statement / feature	Notes
CREATE TABLE [IF NOT EXISTS]	Full schema: INT, DECIMAL, VARCHAR/TEXT, DATETIME; PRIMARY KEY
INSERT (single row)	One VALUES tuple; optional EXPIRES clause
INSERT (batched)	Multiple VALUES tuples in one statement
SELECT * / projected SELECT	Comma-separated column list or *
WHERE (single condition)	= != < > <= >= on any column
INNER JOIN	Two-table equi-join on any column pair
ORDER BY (single column)	ASC or DESC; rows materialised before streaming
DELETE FROM	Full-table delete; WAL-logged
EXPIRES <unix_ms>	Row-level TTL appended to INSERT

6.7 Benchmark results (April 7, 2026, 10 M rows, 4 clients)

Metric	Result
Insert elapsed	9,209 ms
Insert throughput	1,085,894 rows / sec
Full-table SELECT (4 clients)	193,780 ms — 40 M rows streamed total
Primary-key WHERE (4 clients)	26,198 ms
Unit tests	23 / 23 passed

7. Known Limitations and Future Work

- WHERE supports only a single predicate. AND / OR compound conditions require parser and executor changes.
- Secondary indexes are not implemented. Non-PK WHERE clauses always scan.

- ORDER BY and large JOINS materialise full intermediate sets in memory — no external sort.
- No log compaction: WAL replay cost grows linearly with the number of committed operations over the server's lifetime.
- No MVCC: concurrent write serialisation means write throughput is bounded by a single mutex.
- No UPDATE statement: rows can be logically replaced only by DELETE + INSERT.